

Module 2

SystemVerilog Testbench Fundamentals

Learning objectives

- Master SystemVerilog testbench construction

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["SystemVerilog Testbench Arch"]

T2["Clock Generation and Managem"]

T1 --> T2

T3["Reset Patterns and Sequences"]

T2 --> T3

Master SystemVerilog testbench construction

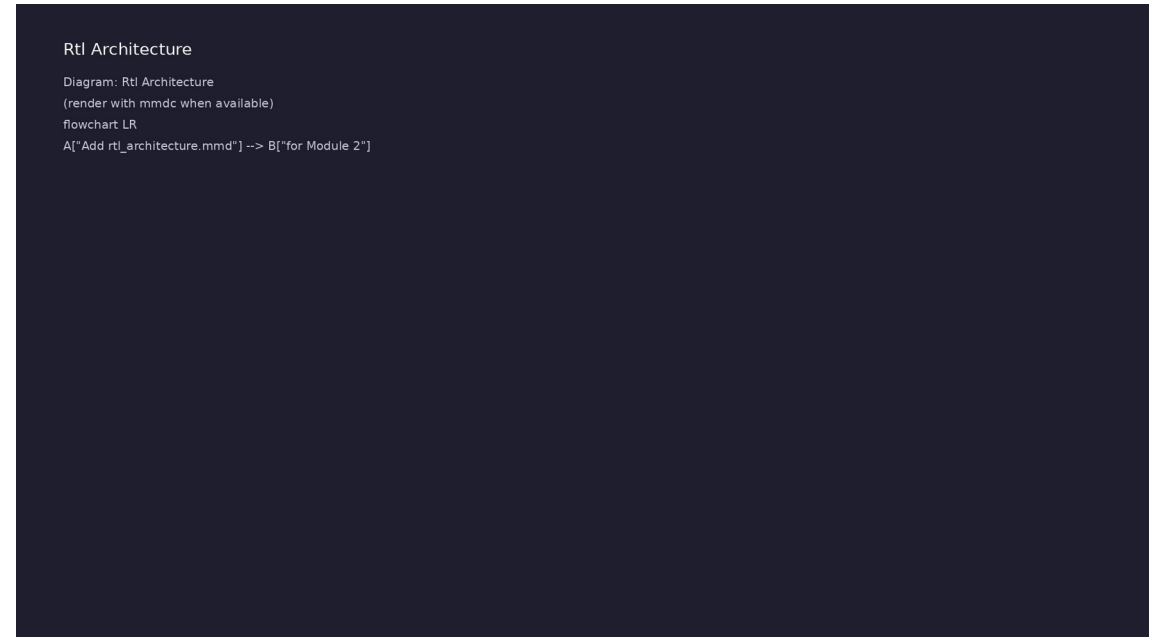
Overview

- This module provides comprehensive coverage of SystemVerilog testbench construction. You'll learn h...

Design architecture

1. SystemVerilog testbench architecture (1/3)

- DUT boundary: signals cross via module ports or SystemVerilog interfaces
- Hierarchy: TB module → DUT instance → optional interface instance
- Time: `#delay`, `@posedge`, and C++ time loop must stay consistent
- See [SYSTEMVERILOG_VERILATOR_INTERACTION.md] (SYSTEMVERILOG_VERILATOR_INTERACTION.md) for compile/si...



1. SystemVerilog testbench architecture (2/3)

- module2/dut/: RTL blocks under test (registers, FIFOs, state machines)
- module2/examples/*/: Focused TB patterns (clock, reset, triggers, access)
- module2/tests/uvm_tests/: UVM wrappers on the same DUTs (bridge to Module 3)
- Testbench module: Instantiates DUT + interfaces; generates clk/rst

1. SystemVerilog testbench architecture (3/3)

- `sim_main.cpp`: C++ entry: drives time, calls `eval()` each cycle

2. RTL block architecture (example DUTs)

- Register files — address decode, read/write timing, reset behavior
- FIFOs — full/empty flags, pointer management, back-pressure
- State machines — explicit states, transition conditions, output logic
- Signal flow: clk/rst → sequential logic → status flags → TB monitors

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR

A["Add verification_architecture.mmd"] --> B["for Module 2"]

3. Verilator execution pipeline

- 1: Makefile — Invokes Verilator with `--cc --exe --timing`
- 2: `obj_dir/V*.cpp` — Generated cycle-accurate C++ model
- 3: `sim_main.cpp` — Clock generation and \$finish coordination
- 4: `./obj_dir/V*` — Executable; TB prints PASS/FAIL

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 2"]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart LR

A["Add rtl_architecture.mmd"] --> B["for Module 2"]

Module 2: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

A["Add verification_architecture.mmd"] --> B["for Module 2"]

Module 2: stimulus, observation, and checking.

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module2 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Artifact flow (spec → sim)

- 1: Makefile — Invokes Verilator with --cc --exe --timing
- 2: obj_dir/V*.cpp — Generated cycle-accurate C++ model
- 3: sim_main.cpp — Clock generation and \$finish coordination
- 4: ./obj_dir/V* — Executable; TB prints PASS/FAIL

Example directory layout

- module2/dut/: RTL blocks under test (registers, FIFOs, state machines)
- module2/examples/*/: Focused TB patterns (clock, reset, triggers, access)
- module2/tests/uvm_tests/: UVM wrappers on the same DUTs (bridge to Module 3)
- Testbench module: Instantiates DUT + interfaces; generates clk/rst
- sim_main.cpp: C++ entry: drives time, calls eval() each cycle

Directed test execution sequence

- make
- link
- run
- grep PASS in log

Verification & testing methods

1. Stimulus and timing strategy

- Clock generation: parameterized period/duty; multiple clocks for CDC awareness
- Reset sequences: assert length, sync deassert, mid-test reset injection
- Triggers: ``@posedge``, ``wait()``, ``fork-join`` for parallel activity
- Signal access: hierarchical references and interface modport access

Testing Methods

Diagram: Testing Methods
(render with mmdc when available)
flowchart LR
A["Add testing_methods.mmd"] --> B["for Module 2"]

2. Checking and debug workflow (1/2)

- Self-checking tests compare DUT outputs against expected values each cycle
- Waveforms (`--trace``) for debugging timing and reset issues
- Closure criteria: all directed cases PASS; no ``$error`` or timeout
- ``./scripts/module2.sh --all-examples`` for module-level regression

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 2"]

2. Checking and debug workflow (2/2)

- UVM tests reuse same DUTs — compare directed vs phase-based execution in Module 3

Syllabus topics

Protocol & design details

Detail: Example 2.1: Clock Generation (`module2/examples/clock_genera...

- Simple Clock: Basic clock generation with `always` block
- Parameterized Clock: Clock with configurable period and duty cycle
- Multiple Clocks: Generating multiple independent clocks
- Clock Gating: Conditional clock generation

Detail: Example 2.1: Clock Generation (`module2/examples/clock_genera...

- Clock Stopping: Controlled clock stopping
- Uses `always` block for continuous execution
- Delay operator `#` controls timing
- Creates 50% duty cycle clock

Detail: Example 2.2: Reset Patterns (`module2/examples/reset_patterns...`)

- Synchronous Reset: Reset synchronized to clock
- Asynchronous Reset: Reset independent of clock
- Reset Sequence: Controlled reset assertion and de-assertion
- Reset Verification: Checking reset behavior

Detail: Example 2.2: Reset Patterns (`module2/examples/reset_patterns...

- Purpose: Encapsulates async reset sequence
- Parameters: Configurable reset duration
- Usage: `async\_reset\_sequence(100);`
- Key Features: Uses delays, displays timing info

Detail: Example 2.3: Signal Access (`module2/examples/signal_access/s...

- Signal Reading: Reading DUT signal values
- Signal Driving: Driving signals to DUT
- Signal Monitoring: Monitoring signal changes
- Signal Types: Handling different signal types

Detail: Example 2.3: Signal Access (`module2/examples/signal_access/s...

- Purpose: Demonstrates reading signal values
- Uses: ``$display()` with format specifiers (``%02h``, ``%0d``, ``%b``)
- Key: Reads signals after delays for proper timing
- Purpose: Demonstrates driving signals to DUT

Detail: Example 2.4: Triggers and Timing (`module2/examples/triggers/...

- Event Control: Using `@` for event-based timing
- Delay Control: Using `#` for time-based delays
- Wait Statements: Using `wait` for condition-based waiting
- Fork-Join: Parallel execution with synchronization

Detail: Example 2.4: Triggers and Timing (`module2/examples/triggers/...

- Named Events: Event-based synchronization
- Timeout Handling: Timeout patterns
- Purpose: Demonstrates event-based timing
- Uses: `@(posedge)`, `@(negedge)`, `@(signal)` for event control

Detail: Example 2.5: Common Patterns (`module2/examples/common_patter...

- Testbench Structure: Organized testbench with phases
- Driver Pattern: Stimulus generation component
- Monitor Pattern: Response monitoring component
- Scoreboard Pattern: Response checking component

Detail: Example 2.5: Common Patterns (`module2/examples/common_patter...

- Reusable Components: Component reuse patterns
- Purpose: Demonstrates organized testbench phases
- Structure: Initialization → Reset → Stimulus →
Checking → Cleanup
- Key: Clear separation of test phases

Detail: Testbench Task Patterns (1)

- Purpose: Encapsulate reset sequences
- Reusability: Can be called from multiple test tasks
- Parameters: Configurable duration/cycles
- Purpose: Generate clock signals

Detail: Testbench Task Patterns (2)

- Pattern: Use ``always`` blocks for continuous generation
- Key: Delay operator ``#`` controls timing
- Purpose: Organize test into phases
- Structure: Clear separation of concerns

Detail: Timing Control Functions and Tasks (1)

- Purpose: Create time-based delays
- Usage: In tasks and initial blocks
- Key: Cannot be used in functions
- Purpose: Synchronize to events

Detail: Timing Control Functions and Tasks (2)

- Usage: In tasks and always blocks
- Key: Event-driven synchronization
- Purpose: Condition-based waiting
- Usage: In tasks

Detail: Built-in Functions for Testbenches (1)

- Purpose: Track simulation time
- Usage: Timing verification, logging
- Purpose: Get signal dimensions
- Usage: Dynamic signal handling

Detail: Built-in Functions for Testbenches (2)

- Purpose: Print formatted output
- Format Specifiers: ``%0d`` (decimal), ``%0h`` (hex),
``%0b`` (binary), ``%0t`` (time)
- Purpose: Verify conditions
- Usage: Test verification

Detail: Common Testbench Patterns (1)

- Pattern: Free-running clock
- Key: Uses ``forever`` loop
- Pattern: Reusable reset sequence
- Key: Encapsulated in task

Detail: Common Testbench Patterns (2)

- Pattern: Loop-based stimulus
- Key: Synchronized to clock
- Pattern: Assertion-based checking
- Key: Uses ``assert`` with ``$error()``

Detail: Best Practices for Testbench Tasks (1)

- Each test phase as separate task
- Helper tasks for common operations
- Clear task naming conventions
- Use parameters for configurability

Detail: Best Practices for Testbench Tasks (2)

- Default parameter values
- Type-safe parameters
- Always synchronize to clock edges
- Use appropriate delays

Detail: Common Pitfalls in Testbench Construction (1)

- Issue: ``forever`` with ``~`` operator can cause timing issues
- Solution: Use explicit assignment with delays
- Issue: Fixed delays don't guarantee clock alignment
- Solution: Always use ``@(posedge clk)`` for synchronization

Detail: Common Pitfalls in Testbench Construction (2)

- Issue: Insufficient reset duration or propagation time
- Solution: Use configurable duration and add propagation delay
- Issue: Non-blocking assignments in tasks can cause timing issues
- Solution: Use blocking assignments (`=`) in tasks

Detail: Performance Considerations (1)

- Simple clocks: Use ``always`` blocks (efficient)
- Multiple clocks: Each clock needs separate ``always`` block
- Gated clocks: Conditional logic adds minimal overhead
- Tasks: Slower due to timing simulation overhead

Detail: Performance Considerations (2)

- Functions: Faster (zero-time), but can't use delays
- Recommendation: Use functions for calculations, tasks for sequences
- ``@(posedge clk)``: Efficient, event-driven
- ``#delay``: Time-based, requires scheduling

Detail: Troubleshooting Guide (1)

- Symptoms: Clock signal stuck at 0 or 1
- Causes:
- `always` block not executing
- Delay value too large or zero

Detail: Troubleshooting Guide (2)

- Blocking on condition that never becomes true
- Solutions:
- Check ``always`` block syntax
- Verify delay values are positive

Verification flow (reference)

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart LR

A["Add testing_methods.mmd"] --> B["for Module 2"]

Stimulus, DUT response, and check points for this module.

1. SystemVerilog Testbench Architecture (1/3)

- What is a Testbench?
- Testbench purpose and structure
- DUT (Design Under Test) connection
- Stimulus generation
- Response checking
- Testbench Components

1. SystemVerilog Testbench Architecture (2/3)

- Testbench module
- DUT instantiation
- Interface instantiation
- Clock and reset generation
- Stimulus and monitoring
- Key Concepts

1. SystemVerilog Testbench Architecture (3/3)

- Module hierarchy
- Signal connectivity
- Time management
- Event scheduling

2. Clock Generation and Management (1/8)

- Clock Generation
- Simple clock generation
- Clock with parameters (period, duty cycle)
- Multiple clocks
- Clock gating
- Clock Patterns

2. Clock Generation and Management (2/8)

- Regular clocks
- Gated clocks
- Clock division
- Clock stopping
- Clock domain crossing
- Clock Domain Management

2. Clock Generation and Management (3/8)

- Multiple clock domains
- Clock domain crossing verification
- Synchronization between domains
- Metastability handling
- Simple Clock: Basic clock generation with ``always`` block
- Parameterized Clock: Clock with configurable period and duty cycle

2. Clock Generation and Management (4/8)

- Multiple Clocks: Generating multiple independent clocks
- Clock Gating: Conditional clock generation
- Clock Stopping: Controlled clock stopping
- Uses ``always`` block for continuous execution
- Delay operator ``#`` controls timing
- Creates 50% duty cycle clock

2. Clock Generation and Management (5/8)

- Uses parameters for flexibility
- Same pattern, different frequency
- Conditional clock generation
- Uses `if-else` for gating control
- Uses `wait` statement for condition-based stopping
- Demonstrates controlled clock stopping

2. Clock Generation and Management (6/8)

- ``$display()` - Print clock information
- ``$time`` - Get current simulation time
- ``$finish`` - Terminate simulation
- Clocks are fundamental to synchronous designs
- Parameterized clocks enable flexibility
- Multiple clocks require careful synchronization

2. Clock Generation and Management (7/8)

- Clock gating is common in low-power designs
- ``always`` blocks create continuous processes
- Delay operator ``#`` controls timing
- ``wait`` statements enable condition-based control
- Simple clock generation with configurable period
- Multiple independent clocks demonstrated

2. Clock Generation and Management (8/8)

- Clock gating and stopping patterns working correctly

3. Reset Patterns and Sequences (1/7)

- Reset Types
 - Synchronous reset
 - Asynchronous reset
- Reset assertion and de-assertion
- Reset timing
- Reset Patterns

3. Reset Patterns and Sequences (2/7)

- Power-on reset
- Software reset
- Hardware reset
- Partial reset
- Reset Sequences
- Reset assertion sequence

3. Reset Patterns and Sequences (3/7)

- Reset de-assertion sequence
- Reset verification
- Reset timing checks
- Synchronous Reset: Reset synchronized to clock
- Asynchronous Reset: Reset independent of clock
- Reset Sequence: Controlled reset assertion and de-assertion

3. Reset Patterns and Sequences (4/7)

- Reset Verification: Checking reset behavior
- Purpose: Encapsulates async reset sequence
- Parameters: Configurable reset duration
- Usage: ``async_reset_sequence(100);``
- Key Features: Uses delays, displays timing info
- Purpose: Encapsulates sync reset sequence

3. Reset Patterns and Sequences (5/7)

- Parameters: Number of reset cycles
- Usage: ``sync_reset_sequence(5);``
- Key Features: Uses ``repeat`` and ``@(posedge clk)`` for clock synchronization
- Purpose: Verifies reset properly initializes registers
- Uses: ``assert`` statements with ``$error()`` for verification
- Usage: ``verify_reset();``

3. Reset Patterns and Sequences (6/7)

- Reset is critical for proper design initialization
- Synchronous vs asynchronous reset have different timing
- Reset sequences must follow design requirements
- Reset verification ensures proper operation
- Tasks encapsulate reset sequences for reusability
- ``repeat`` statement repeats clock cycles

3. Reset Patterns and Sequences (7/7)

- ``@(posedge clk)`` synchronizes to clock edges
- ``assert`` statements verify reset behavior
- Synchronous and asynchronous reset patterns demonstrated
- Reset sequences executed correctly
- Reset verification checks passing

4. Signal Access and Driving (1/8)

- Signal Access
- Reading signal values
- Signal types and widths
- Signal state checking
- Signal monitoring
- Driving Signals

4. Signal Access and Driving (2/8)

- Assigning values
- Continuous assignment
- Procedural assignment
- Signal driving timing
- Signal Types
- Single-bit signals

4. Signal Access and Driving (3/8)

- Multi-bit signals (vectors)
- Buses and arrays
- Bidirectional signals
- Signal Reading: Reading DUT signal values
- Signal Driving: Driving signals to DUT
- Signal Monitoring: Monitoring signal changes

4. Signal Access and Driving (4/8)

- Signal Types: Handling different signal types
- Purpose: Demonstrates reading signal values
- Uses: ``$display()` with format specifiers (``%02h``, ``%0d``, ``%b``)
- Key: Reads signals after delays for proper timing
- Purpose: Demonstrates driving signals to DUT
- Uses: Blocking assignment (``=``) for signal driving

4. Signal Access and Driving (5/8)

- Verification: Uses ``assert`` to verify driven values
- Purpose: Automatically monitors signal changes
- Uses: ``always @(signal)`` for event-driven monitoring
- Key: Runs in parallel with test tasks
- Purpose: Demonstrates different signal types
- Uses: ``$bits()`` built-in function for signal width

4. Signal Access and Driving (6/8)

- Key: Handles single-bit, multi-bit, and array signals
- ``$display()`` - Print signal values with formatting
- ``$time`` - Get current simulation time
- ``$bits()`` - Get signal bit width
- ``$error()`` - Report errors in assertions
- Signal access is fundamental to testbench operation

4. Signal Access and Driving (7/8)

- Proper timing is essential for signal driving
- Signal monitoring enables response checking
- Different signal types require different handling
- Tasks organize signal access operations
- ``always @(signal)`` creates event-driven monitors
- Blocking assignment (``=``) for signal driving

4. Signal Access and Driving (8/8)

- Format specifiers (`%b`, `%h`, `%d`) for display
- Signal reading operations working correctly
- Signal driving verified with assertions
- Signal monitoring capturing changes
- Different signal types handled properly

5. Event and Timing Control (1/9)

- SystemVerilog Events
- Event declaration
- Event triggering (``->``)
- Event waiting (``@``)
- Named events
- Timing Control

5. Event and Timing Control (2/9)

- `#` delay operator
- `@` event control
- `wait` statement
- `fork-join` constructs
- Synchronization
- Task synchronization

5. Event and Timing Control (3/9)

- Barrier synchronization
- Timeout handling
- Race condition avoidance
- Event Control: Using ``@`` for event-based timing
- Delay Control: Using ``#`` for time-based delays
- Wait Statements: Using ``wait`` for condition-based waiting

5. Event and Timing Control (4/9)

- Fork-Join: Parallel execution with synchronization
- Named Events: Event-based synchronization
- Timeout Handling: Timeout patterns
- Purpose: Demonstrates event-based timing
- Uses: ``@(posedge)``, ``@(negedge)``, ``@(signal)`` for event control
- Key: Waits for specific events before continuing

5. Event and Timing Control (5/9)

- Purpose: Demonstrates time-based delays
- Uses: ``#`` delay operator with fixed and parameterized values
- Key: Tracks time using ``$time`` built-in function
- Purpose: Demonstrates condition-based waiting
- Uses: ``wait(condition)`` statement
- Key: Waits until condition becomes true

5. Event and Timing Control (6/9)

- Purpose: Demonstrates parallel execution
- Uses: ``fork-join`` for parallel tasks
- Key: Waits for all tasks to complete
- Purpose: Continues after first task completes
- Uses: ``fork-join_any`` construct
- Key: Doesn't wait for all tasks

5. Event and Timing Control (7/9)

- Purpose: Starts tasks without waiting
- Uses: ``fork-join_none`` construct
- Key: Continues immediately, tasks run in background
- Purpose: Demonstrates named event synchronization
- Uses: ``event`` declaration, ``@(event)`` wait, ``->event`` trigger
- Key: Flexible synchronization mechanism

5. Event and Timing Control (8/9)

- Events enable synchronization between processes
- Timing control is essential for proper testbench operation
- Wait statements enable condition-based synchronization
- Fork-join enables parallel execution
- ``@(posedge/negedge)`` waits for signal edges
- ``#`` delay operator creates time-based delays

5. Event and Timing Control (9/9)

- ``wait(condition)`` waits for condition to become true
- ``fork-join`` creates parallel execution with synchronization
- Named events provide flexible synchronization

6. Common Verification Patterns (1/7)

- Testbench Structure
- Initialization phase
- Stimulus generation
- Response checking
- Cleanup phase
- Verification Patterns

6. Common Verification Patterns (2/7)

- Directed testing
- Constrained random testing
- Coverage-driven verification
- Assertion-based verification
- Reusable Components
- Driver components

6. Common Verification Patterns (3/7)

- Monitor components
- Scoreboard components
- Checker components
- Testbench Structure: Organized testbench with phases
- Driver Pattern: Stimulus generation component
- Monitor Pattern: Response monitoring component

6. Common Verification Patterns (4/7)

- Scoreboard Pattern: Response checking component
- Reusable Components: Component reuse patterns
- Purpose: Demonstrates organized testbench phases
- Structure: Initialization → Reset → Stimulus →
Checking → Cleanup
- Key: Clear separation of test phases
- Purpose: Stimulus generation component

6. Common Verification Patterns (5/7)

- Uses: Virtual task for polymorphism
- Key: Can be extended in derived classes
- Purpose: Response monitoring component
- Uses: Constructor function, virtual task, accessor function
- Key: Captures and provides access to monitored data
- Purpose: Response checking component

6. Common Verification Patterns (6/7)

- Uses: Queue methods (``push_back()``, ``size()``), comparison logic
- Key: Compares expected vs actual results
- Structured testbenches are easier to maintain
- Reusable components improve productivity
- Common patterns enable code reuse
- Patterns prepare for UVM methodology

6. Common Verification Patterns (7/7)

- Tasks organize test phases (init, reset, stimulus, check)
- Virtual tasks enable polymorphism
- Class methods encapsulate component behavior
- Queue operations manage transaction lists

7. Test Structure and Organization (1/3)

- Test Function Structure
- Test module organization
- Test phases
- Test organization
- Test parameters
- Multiple Tests

7. Test Structure and Organization (2/3)

- Test discovery
- Test selection
- Test parameters
- Test fixtures
- Test Organization
- Test file structure

7. Test Structure and Organization (3/3)

- Test naming conventions
- Test documentation
- Test maintenance

7. Functions and Tasks for Testbench Construction

(1/23)

- Purpose: Encapsulate reset sequences
- Reusability: Can be called from multiple test tasks
- Parameters: Configurable duration/cycles
- Purpose: Generate clock signals
- Pattern: Use ``always`` blocks for continuous generation
- Key: Delay operator ``#`` controls timing

7. Functions and Tasks for Testbench Construction

(2/23)

- Purpose: Organize test into phases
- Structure: Clear separation of concerns
- Maintainability: Easy to modify individual phases
- Purpose: Encapsulate signal operations
- Output Parameters: Use `output` for return values
- Timing: Synchronize with clock edges

7. Functions and Tasks for Testbench Construction

(3/23)

- Purpose: Create time-based delays
- Usage: In tasks and initial blocks
- Key: Cannot be used in functions
- Purpose: Synchronize to events
- Usage: In tasks and always blocks
- Key: Event-driven synchronization

7. Functions and Tasks for Testbench Construction

(4/23)

- Purpose: Condition-based waiting
- Usage: In tasks
- Key: Waits until condition becomes true
- Purpose: Parallel execution
- Usage: In tasks
- Key: Different join types for different synchronization needs

7. Functions and Tasks for Testbench Construction

(5/23)

- Purpose: Track simulation time
- Usage: Timing verification, logging
- Purpose: Get signal dimensions
- Usage: Dynamic signal handling
- Purpose: Print formatted output
- Format Specifiers: ``%0d`` (decimal), ``%0h`` (hex),
``%0b`` (binary), ``%0t`` (time)

7. Functions and Tasks for Testbench Construction

(6/23)

- Purpose: Verify conditions
- Usage: Test verification
- Severity: ERROR, FATAL, WARNING
- Pattern: Free-running clock
- Key: Uses ``forever`` loop
- Pattern: Reusable reset sequence

7. Functions and Tasks for Testbench Construction

(7/23)

- Key: Encapsulated in task
- Pattern: Loop-based stimulus
- Key: Synchronized to clock
- Pattern: Assertion-based checking
- Key: Uses ``assert`` with ``$error()``
- Pattern: Event-driven monitoring

7. Functions and Tasks for Testbench Construction

(8/23)

- Key: Uses ``always @(posedge clk)``
- Each test phase as separate task
- Helper tasks for common operations
- Clear task naming conventions
- Use parameters for configurability
- Default parameter values

7. Functions and Tasks for Testbench Construction

(9/23)

- Type-safe parameters
- Always synchronize to clock edges
- Use appropriate delays
- Avoid race conditions
- Use assertions for verification
- Provide informative error messages

7. Functions and Tasks for Testbench Construction

(10/23)

- Handle edge cases
- Encapsulate common sequences
- Use virtual tasks for polymorphism
- Create reusable components
- Issue: ``forever`` with ``~`` operator can cause timing issues
- Solution: Use explicit assignment with delays

7. Functions and Tasks for Testbench Construction

(11/23)

- Issue: Fixed delays don't guarantee clock alignment
- Solution: Always use ``@(posedge clk)`` for synchronization
- Issue: Insufficient reset duration or propagation time
- Solution: Use configurable duration and add propagation delay
- Issue: Non-blocking assignments in tasks can cause timing issues
- Solution: Use blocking assignments (``=``) in tasks

7. Functions and Tasks for Testbench Construction

(12/23)

- Issue: No way to verify tasks completed successfully
- Solution: Add completion messages or flags
- Issue: Wait statement can hang if condition never becomes true
- Solution: Use fork-join with timeout
- Simple clocks: Use `always` blocks (efficient)
- Multiple clocks: Each clock needs separate `always` block

7. Functions and Tasks for Testbench Construction

(13/23)

- Gated clocks: Conditional logic adds minimal overhead
- Tasks: Slower due to timing simulation overhead
- Functions: Faster (zero-time), but can't use delays
- Recommendation: Use functions for calculations, tasks for sequences
- ``@(posedge clk)``: Efficient, event-driven
- ``#delay``: Time-based, requires scheduling

7. Functions and Tasks for Testbench Construction

(14/23)

- ``wait(condition)``: Polls condition, can be slower
- ``fork-join``: Creates parallel processes (overhead)
- Use sparingly: Only when parallel execution is needed
- Alternative: Sequential execution is faster
- Symptoms: Clock signal stuck at 0 or 1
- Causes:

7. Functions and Tasks for Testbench Construction

(15/23)

- ``always`` block not executing
- Delay value too large or zero
- Blocking on condition that never becomes true
- Solutions:
- Check ``always`` block syntax
- Verify delay values are positive

7. Functions and Tasks for Testbench Construction

(16/23)

- Use ``$display`` to debug clock generation
- Symptoms: Registers not resetting to expected values
- Causes:
 - Insufficient reset duration
 - Reset de-asserted too early
 - Missing propagation delay

7. Functions and Tasks for Testbench Construction

(17/23)

- Solutions:
- Increase reset duration
- Add delay after reset de-assert
- Verify reset polarity (active-high vs active-low)
- Symptoms: Signal value doesn't change as expected
- Causes:

7. Functions and Tasks for Testbench Construction

(18/23)

- Not synchronized to clock
- Using wrong assignment type
- Signal not being driven
- Solutions:
- Use ``@(posedge clk)`` before reading
- Use blocking assignment (``=``) in tasks

7. Functions and Tasks for Testbench Construction

(19/23)

- Verify signal is being driven
- Symptoms: Inconsistent behavior, timing-dependent failures
- Causes:
 - Multiple processes accessing same signal
 - Clock and data changing simultaneously
 - Missing synchronization

7. Functions and Tasks for Testbench Construction

(20/23)

- Solutions:
- Use proper clock synchronization
- Avoid shared variable access
- Use non-blocking assignment in sequential logic
- Symptoms: Code continues before tasks complete
- Causes:

7. Functions and Tasks for Testbench Construction (21/23)

- Using ``join_none`` instead of ``join``
- Tasks completing before join
- Solutions:
- Use ``join`` to wait for all tasks
- Use ``join_any`` to wait for first task
- Add completion flags if needed

7. Functions and Tasks for Testbench Construction

(22/23)

- Symptoms: Simulation hangs, never completes
- Causes:
 - Condition never becomes true
 - Signal never assigned
 - Logic error in condition
- Solutions:

7. Functions and Tasks for Testbench Construction (23/23)

- Add timeout mechanism
- Verify condition logic
- Check signal assignments
- Use ``$display`` to debug

Hands-on examples

Module 2 self-check

```
$ ./scripts/module2.sh --help
```

Terminal: module2_check

Usage: ./scripts/module2.sh [OPTIONS]

Module 2: SystemVerilog Testbench Fundamentals

This script runs examples and tests for Module 2.

OPTIONS:

SystemVerilog Examples:

--clock-generation	Run clock generation examples
--reset-patterns	Run reset patterns examples
--signal-access	Run signal access examples
--triggers	Run triggers and timing control examples
--common-patterns	Run common verification patterns examples
--all-examples	Run all SystemVerilog examples (default)
--skip-examples	Skip all SystemVerilog examples

Tests:

--uvm-tests	Run UVM testbenches
--all-tests	Run all tests

Environment:

--sim SIMULATOR	Simulator to use (default: verilator)
--uvm-home DIR	UVM library directory (default: \$UVM_HOME)
--jobs N	Number of parallel build jobs (default: 8)
--no-clean	Skip cleaning before build (faster rebuilds)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all SystemVerilog examples (with parallel builds)
./scripts/module2.sh
```

```
# Run only clock generation
./scripts/module2.sh --clock-generation
```

```
# Run UVM tests
```

Exercise scaffold

```
./scripts/module2.sh --scaffold
```

Example 1: Clock Generation

- Simple Clock: Basic clock generation with ``always`` block
- Parameterized Clock: Clock with configurable period and duty cycle
- Multiple Clocks: Generating multiple independent clocks
- Clock Gating: Conditional clock generation
- Clock Stopping: Controlled clock stopping

Demo: Clock Generation

```
$ ./scripts/module2.sh --clock-generation
```

Terminal: ex01_clock_generation

```
[0:36m=====0m
[0:36mModule 2: SystemVerilog Testbench Fundamentals0m
[0:36m=====0m

[0:34m[2026-05-31 02:35:02] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 02:35:02] Checking prerequisites...0m
[0:31m[2026-05-31 02:35:02] Error: verilator not found. Please install it first.0m
```

Example 2: Reset Patterns

- Synchronous Reset: Reset synchronized to clock
- Asynchronous Reset: Reset independent of clock
- Reset Sequence: Controlled reset assertion and de-assertion
- Reset Verification: Checking reset behavior

Demo: Reset Patterns

```
$ ./scripts/module2.sh --reset-patterns
```

Terminal: ex02_reset_patterns

```
[0:36m=====
[0:36mModule 2: SystemVerilog Testbench Fundamentals
[0:36m=====

[0:34m[2026-05-31 02:35:03] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 02:35:03] Checking prerequisites...
[0:31m[2026-05-31 02:35:03] Error: verilator not found. Please install it first.
```

Example 3: Signal Access

- Signal Reading: Reading DUT signal values
- Signal Driving: Driving signals to DUT
- Signal Monitoring: Monitoring signal changes
- Signal Types: Handling different signal types

Demo: Signal Access

```
$ ./scripts/module2.sh --signal-access
```

Terminal: ex03_signal_access

```
[0:36m=====
[0:36mModule 2: SystemVerilog Testbench Fundamentals
[0:36m=====

[0:34m[2026-05-31 02:35:03] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 02:35:03] Checking prerequisites...
[0:31m[2026-05-31 02:35:04] Error: verilator not found. Please install it first.
```


Example 4: Triggers and Timing

- Event Control: Using ``@`` for event-based timing
- Delay Control: Using ``#`` for time-based delays
- Wait Statements: Using ``wait`` for condition-based waiting
- Fork-Join: Parallel execution with synchronization
- Named Events: Event-based synchronization

Demo: Triggers and Timing

```
$ ./scripts/module2.sh --triggers
```

Terminal: ex04_triggers

```
[0:36m=====0m
[0:36mModule 2: SystemVerilog Testbench Fundamentals0m
[0:36m=====0m

[0:34m[2026-05-31 02:35:04] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 02:35:04] Checking prerequisites...0m
[0:31m[2026-05-31 02:35:04] Error: verilator not found. Please install it first.0m
```

Example 5: Common Patterns

- Testbench Structure: Organized testbench with phases
- Driver Pattern: Stimulus generation component
- Monitor Pattern: Response monitoring component
- Scoreboard Pattern: Response checking component
- Reusable Components: Component reuse patterns

Demo: Common Patterns

```
$ ./scripts/module2.sh --common-patterns
```

Terminal: ex05_common_patterns

```
[0:36m=====
[0:36mModule 2: SystemVerilog Testbench Fundamentals
[0:36m=====

[0:34m[2026-05-31 02:35:05] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0:34m[2026-05-31 02:35:05] Checking prerequisites...
[0:31m[2026-05-31 02:35:05] Error: verilator not found. Please install it first.
```

Common pitfalls

Clock Generation

- Create a parameterized clock generator
- Generate multiple clocks with different frequencies
- Implement clock gating
- Test clock domain crossing

Reset Implementation

- Implement synchronous and asynchronous reset
- Create reset sequences
- Verify reset behavior
- Test reset timing

Signal Access

- Read and drive DUT signals
- Monitor signal changes
- Handle different signal types
- Implement signal checking

Timing Control

- Use events for synchronization
- Implement delays and waits
- Use fork-join for parallel execution
- Handle timeouts

Testbench Patterns

- Implement driver pattern
- Implement monitor pattern
- Implement scoreboard pattern
- Create reusable testbench components

Practice & assessment

What you should know (1/2)

- Understand SystemVerilog testbench architecture
- Generate clocks and manage clock domains
- Implement reset patterns and sequences
- Access and drive signals correctly
- Use events and timing control effectively
- Apply common verification patterns

What you should know (2/2)

- Structure testbenches for maintainability
- Prepare for UVM methodology
- Write effective testbench tasks and functions
- Use timing control constructs properly

Exercises

- Clock Generation
- Reset Implementation
- Signal Access
- Timing Control
- Testbench Patterns

Assessment checklist

- Can generate clocks and manage clock domains
- Can implement reset patterns
- Can access and drive signals correctly
- Can use events and timing control
- Can apply common verification patterns
- Can structure testbenches effectively

Summary & next steps

- Master SystemVerilog testbench construction
- Complete module2/CHECKLIST.md
- Review module2/EXAMPLES.md and run each lab
- Next: Next module in course